



Practica 1

Analizadores léxicos con Lex (Flex)

No. de cuenta	Nombre
321225087	Rivera Jimenez Isaac

Compiladores

Semestre 2027-1



Consideraciones

En la carpeta *Lex_Primer_programa* se encuentra el primer archivo lex llamado *archivo.ll*. Y el archivo modificado llamado *archivoNuevo.lex*

Pasos

- Transcribir el código anterior a un archivo con extensión *.ll*, *.lex* o *.flex* dentro de la carpeta *src/Primer programa* en *Lex/*.
- Compilar mediante la instrucción: `flex++ archivo.ll`
- Comprobar que se generó el archivo `lex.yy.cc`.
- Compilar mediante: `g++ lex.yy.cc -o nombreEjecutable`
- Ejecutar mediante: `./nombreEjecutable`

```
%{
    #include <iostream>
}%

%option c++
%option noyywrap

digito [0-9]
letra [a-zA-Z]
palabra {letra}+
espacio [ \t\n]

%%

{espacio} { /* La acción léxica puede ir vacía si queremos que el escáner ignore la regla */
{digito}+ { std::cout << "Encontré un número: " << yytext << std::endl; }
{palabra} { std::cout << "Encontré una palabra: " << yytext << std::endl; }

%%

int main() {
    FlexLexer* lexer = new yyFlexLexer;
    lexer->yylex();
}
```

El archivo que se creo fue *program.c* por accidente, por lo tanto al referirnos a *program.c* nos referimos a *programa.c*, por lo tanto los archivos generados siguen la misma secuencia.

- ¿Qué ocurre si en la primera sección se quitan las llaves al nombre de la macro `letra`?
(0.5 pts)

Al escribir `palabra letra+`, Flex no sustituye la macro por su definición (`[a-zA-Z]`). En su lugar, lo interpreta como texto, he intenta buscar realmente la palabra *letra* seguida de una o más letras "a".

2. ¿Qué ocurre si en la segunda sección se quitan las llaves a las macros? **(0.5 pts)**

Al quitar las llaves por ejemplo, escribir `palabra` en lugar de `{palabra}`, Flex deja de interpretar la macro como una cadena de texto literal.

Como consecuencia, al ingresar palabras comunes en la consola, estas ya no coinciden con la regla y el la parte de código que menciona:

```
std::cout << "Encontré una palabra: " << yytext << std::endl;
```

no llega a ejecutarse, lo que provoca que no se muestre el mensaje con `yytext` y se active la regla por defecto. La acción solo se ejecutará si el usuario introduce exactamente el texto literal "palabra" o "dígito".

3. ¿Cómo se escribe un comentario en flex? **(0.5 pts)**

La escritura de los comentarios sigue una sintaxis similar a C/C++ utilizando `/* ... */` o `// ...`. Sin embargo, en Flex hay una regla, los comentarios deben estar indentados (iniciar con al menos un espacio o tabulación) o colocarse dentro del bloque de acción entre llaves, por ejemplo: `{ /* cualquier comentario */ }`.

Si se escribe un comentario sin indentar al inicio de una línea en la sección de reglas, Flex lo interpretara como una expresión regular en lugar de un comentario.[1]

4. ¿Qué se guarda en `yytext`? **(0.5 pts)**

Dentro de `yytext` se toma como una variable en donde Flex guarda temporalmente el lema exacto de la entrada que coincidió con la regla léxica actual. Es decir que si el analizador detecta un número como 5, `yytext` almacenará la cadena "5". Por lo tanto se puede considerar como un puntero de tipo `char*`

5. ¿Qué pasa al ejecutar el programa e introducir cadenas de caracteres y de dígitos por la consola? **(0.5 pts)**

El programa los reconoce adecuadamente, lo que permite procesar los lexemas que recibe de la entrada como por ejemplo número (`{dígito}+`) y palabra (`{palabra}`) para reconocer e identificar los tokens correspondientes. Después de analizar los caracteres, imprime mensajes específicos según la regla con la que coincidan. Si se ingresan dígitos, ejecuta la acción asociada y muestra: `'Encontré un número: <dígitos>'`. Si se ingresan secuencias de letras, muestra: `'Encontré una palabra: <cadena>'`. Además, la regla `{espacio}` se encarga de ignorar los espacios, tabulaciones y saltos de línea.

6. ¿Qué ocurre si introducimos caracteres como `"*"` en la consola? **(0.5 pts)**

Simplemente devuelve o imprime en pantalla el símbolo de entrada, sin mostrar ningún mensaje de reconocimiento. Esto sucede porque este tipo de caracteres no están especificados en las expresiones regulares de la segunda sección. Al no estar definido en ninguna

regla, Flex aplica una regla por defecto, la cual toma cualquier carácter no reconocido y lo copia directamente a la salida estándar sin especificar qué fue lo que leyó.

7. Modificar el código anterior en un archivo nuevo, de tal manera que reconozca lo siguiente: **(2 pts)**

- La expresión regular para los hexadecimales en lenguaje C++.
Para esto se formo un nuevo lexema, en donde se tomara en cuenta los hexadecimales con la expresión regular `0[xX][0-9a-fA-F]+`, donde se considera `x` o `X`
- 5 palabras reservadas del lenguaje C++.
Para esto tomamos las palabras `while`, `for`, `int`, `string`, `if`, dentro de la segunda sección de definiciones de Flex, con la palabra reservada.
- Los identificadores válidos del lenguaje C++, con longitud máxima de 32 caracteres (Sugerencia: use el operador `{m,n}`).
Se hizo uso de una longitud máxima de 32 caracteres, mínima de 0 para considerar 'a,b,c...' como una variable, al igual que utilizamos un guion bajo como parte de los identificadores.
- Los espacios en blanco.
El ejercicio iv esta completado en la palabra reservada 'espacio'*/

Analizador léxico para el lenguaje C_1

8. Describir el conjunto de terminales y la expresión regular que reconoce a cada uno en `lexer.ll`. (2 pts)

Reutilizando parte de las expresiones regulares definidas anteriormente, podemos hacer que reconozca a cada uno en `lexer.ll`. Solo realizamos un cambio para el numero decimal, pues dado que necesitamos numero con o sin punto decimal hacemos uso de `?` para demarcar una o ninguna instancia, ademas que consideramos no poder empezar con `n` cantidad de 0's. Finalmente, en la parte del exponente lo delimitamos a solo colocar un dígito, como fue propuesto en las salidas esperadas.

```
DIG [0-9]
/*Reutilizamos las expresiones anteriores*/
letra [a-zA-Z]
palabra {letra}+
espacio [ \t\n]
identificador [a-zA-Z_][a-zA-Z0-9_]{0,31}
decimal {DIG}+(\.{DIG}+)?([e][+-]?{DIG})?
```

Para las palabras reservadas, signos de puntuacion y operadores los asignamos de la siguiente manera:

```
/*Colocamos las palabras reservadas, signos y operadores*/
mas "+"
meno "-"
```

```

multi "*"
div "\\"
asig "="
lr "("
rr ")"
coma ","
pycoma ";"
if "if"
int "int"
while "while"
else "else"
float "float"

```

9. Generar acciones léxicas para cada terminal de nuestro lenguaje en Lexer.cpp, de modo que se muestre en pantalla la salida esperada con el archivo prueba. (2 pts)

Las acciones léxicas son fragmentos de código en lenguaje C que se asocian a patrones específicos, como expresiones regulares; dentro de herramientas generadoras de analizadores léxicos, en este caso Flex.

En este caso para la segunda sección se retorna el número al que fue asociado cada token, de esta forma aseguramos que la salida esperada sea la correcta respecto a cada token que se solicita.

```

%%

/*retornamos el numero al que fue asociado el token*/
{espacio}
{lr} {return 6;}
{rr} {return 7;}
{coma} {return 8;}
{pycoma} {return 9;}
{if} {return 11;}
{int} {return 12;}
{while} {return 13;}
{else} {return 14;}
{float} {return 15;}
{identificador} {return 10;}
{decimal} {return 16;}
. { cout << "ERROR LEXICO" << yytext << endl;}

%%

```

10. Crear un Makefile. (1 pt)

Un make es una utilidad disponible en Linux que agiliza la tarea de compilar código desde la terminal. Nos evita tener que escribir los comandos de compilación a mano, que suelen ser muy largos, y en cambio nos permite escribir algo mucho más corto que al final hace lo mismo.

Se presentan generalmente secciones importante como la definición de variables que se centralizan las herramientas y banderas (CXX = g++, FLEX = flex, CXXFLAGS = -Wall -std=c++11, TARGET = compiler). La generación del scanner que se considera como la regla que detecta cambios en lexer.ll y ejecuta flex lexer.ll para traducir a código fuente. La compilación del binario es la regla que depende de Lexer.cpp y main.cpp, invocando a g++ para compilar ambos archivos y generar el ejecutable final ./compiler.[3][2]

Referencias

- [1] Oracle Corporation. *x86 Assembly Language Reference Manual*. Oracle Documentation, Part No. 802-5880. Disponible en: <https://docs.oracle.com/cd/E19504-01/802-5880/6i9k05dggk/index.html>.
- [2] Z. Xu. *A Brief Introduction to Makefile*. Department of Applied and Computational Mathematics and Statistics, University of Notre Dame. Disponible en: <https://academicweb.nd.edu/~zxu2/acms60212-40212/Makefile.pdf>.
- [3] B. A. Maxwell. *A Simple Makefile Tutorial*. Department of Computer Science, Colby College. Disponible en: <https://www.cs.colby.edu/maxwell/courses/tutorials/maketutor/>.